

SKALA

모델 서빙 및 AIOps 구성

2026.9

임성열 교수 Ph.D

SK 주식회사 AX



1. 모델 서빙 기초 (FastAPI)
2. MLOps 파이프라인 (컨테이너화)
3. AIOps 파이프라인 (모니터링·자동대응)

1. 모델 서빙 기초

- 모델 서빙, AIOps 이해
- 모델 서빙 아키텍처
- FastAPI 기초
- 비동기 처리
- 서빙 안정성
- FastAPI 엔드 포인트 실습
- FastAPI 서버 빌드업 실습

SK 주식회사 AX

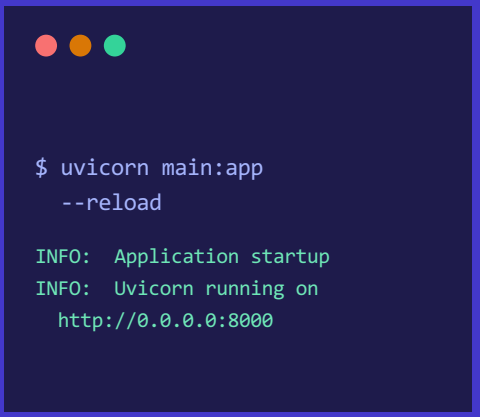
모델 서빙, AIOps 이해

DAY 1

모델 서빙 및 AIOps 구성

FastAPI로 추론 서버를 직접 구축하는 서빙 기초 과정

교육 기간	**3일 · 24시간 (Day1)**
교육 방식	**이론 50% · 실습 50%**
Day1 목표	**FastAPI 추론 서버 구축**



24H PROGRAM

Day1 → Day2 → Day3

서빙 → 파이프라인 → 운영

모델 서빙, AIOps 이해

COURSE :: OVERVIEW

교육 개요

교육 목표

- 모델을 API 형태로 서빙하는 기본 구조를 이해하고, FastAPI로 추론 서버를 직접 구축
- 모델을 안정적으로 버전 관리·배포할 수 있는 파이프라인 구조를 이해하고, 컨테이너화까지 실습 수행
- 서빙 중인 모델의 상태를 모니터링하고, 이상 상황에 자동으로 대응하는 운영 체계 이해

3일 로드맵

Day 1 모델 서빙 기초 (FastAPI)
Day 2 MLOps 파이프라인 (컨테이너화)
Day 3 AIOps 파이프라인 (모니터링·자동대응)

교육 기간
3일, 24시간

교육 방식
이론 50% · 실습 50%

필수 선행 과목
없음

필수 후행 과목
없음

Day1 핵심 산출물

FastAPI 기반 추론 서버 구축 완료 - 라우팅·스키마검증·에러핸들링·헬스체크·Swagger 문서화 포함

모델 서빙, AIOps 이해

DAY1 :: AGENDA

Day 1 목차 한눈에 보기

목차	주요 학습내용	시간	방식
모델 서빙, AIOps 이해	AI 서비스에서 모델 서빙과 AIOps 위상 및 제공 가치	1h	이론
모델 서빙 아키텍처	학습 vs 서빙, 배치 vs 실시간 추론, 서빙 패턴(REST/gRPC, 동기/비동기)	1h	이론
FastAPI 기초	라우팅·요청/응답 모델, Pydantic 스키마 정의, 자동 문서화(Swagger)	1h	이론
비동기 처리	async/await, 동시 요청 처리, 배치 처리로 처리량 개선, 동기 vs 비동기 서버 비교	1h	이론
서빙 안정성	추론 서버 구현, 입력 검증/에러 핸들링, 헬스체크, Lazy/Eager 로딩, 로깅 기초	1h	이론/실습
FastAPI 추론 엔드포인트 실습	간단한 ML/LLM 모델을 FastAPI 엔드포인트로 래핑, 요청→응답 흐름 구현	1h	개념실습
추론 서버 빌드업 통합 실습	라우팅+스키마+에러핸들링+헬스체크 통합, Swagger 테스트, Lazy vs Eager 실습	2h	종합실습

모델 서빙, AIOps 이해

DAY1 :: PIPELINE STAGE 1/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

모델 서빙, AIOps 이해

--theory · 1h

- AI 서비스에서 '모델 서빙(Serving)'이 차지하는 위상 - 학습된 모델을 실제 사용자 요청에 응답시키는 단계
- 모델 서빙과 AIOps가 제공하는 가치 - 안정적 응답, 확장 가능한 인프라, 운영 자동화
- 이번 3일 과정 전체에서 Day1(서빙)→Day2(파이프라인)→Day3(운영)로 이어지는 흐름 소개
- [여기에 상세 내용 및 사례를 채워 넣으세요]

모델 서빙, AIOps 이해

DAY1 :: PIPELINE STAGE 1/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

AI 서비스 파이프라인에서 서빙의 위치

핵심 개념

데이터 수집·전처리 → 모델 학습(Training) → 모델 아티팩트 → **모델 서빙(Serving)** → 모니터링 (AIOps)

- 서빙(Serving)은 학습이 끝난 모델을 실제 요청에 응답시키는 '운영' 단계 - 이번 Day1의 범위
- 서빙된 모델은 이후 지속적으로 모니터링되어야 하며, 이 운영 자동화가 AIOps(Day3)의 영역
- 즉 이번 3일 과정은 이 파이프라인의 뒷부분(서빙→모니터링)을 순서대로 다룸

모델 서빙, AIOps 이해

DAY1 :: PIPELINE STAGE 1/7

서빙이 어려운 이유 - 학습과 다른 요구사항

핵심 개념

구분	학습 (Training)	서빙 (Serving)
실행 주기	1회성 배치 작업 (수 시간~수 일)	24시간 상시 응답 (수 밀리초 단위)
핵심 지표	정확도(Accuracy), Loss	지연시간(Latency), 처리량(Throughput), 가용성
실패 시 영향	재학습하면 됨	사용자 요청이 즉시 실패 - 서비스 장애로 직결
자원 사용 패턴	예측 가능한 배치 부하	예측 불가능한 실시간 트래픽 변동

모델 서빙, AIOps 이해

DAY1 :: PIPELINE STAGE 1/7

① 서빙·AIOps이해 | ② 서빙아키텍처 | ③ FastAPI기초 | ④ 비동기처리 | ⑤ 서빙안정성 | ⑥ 엔드포인트실습 | ⑦ 통합실습

AIOps가 제공하는 운영 가치

핵심 개념

- 자동 모니터링 - 지연시간·에러율·처리량을 사람이 계속 지켜보지 않아도 자동 수집
- 이상 탐지(Anomaly Detection) - 평소와 다른 패턴(느려짐, 에러 급증)을 자동으로 감지
- 자동 대응(Auto Remediation) - 임계값 초과 시 알림 발송 또는 재시작·롤백 등 자동 조치

Day1에서 만든 서빙 서버가 Day3에서 이 AIOps 체계의 '모니터링 대상'이 됩니다. 지금 만드는 헬스체크·로깅이 그 기반입니다.

모델 서빙 아키텍처

DAY1 :: PIPELINE STAGE 2/7

--theory · 1h

- 학습(Training) vs 서빙(Serving)의 근본적 차이 - 목적, 실행 주기, 성능 지표가 다름
- 배치 추론(Batch Inference) vs 실시간 추론(Real-time Inference) 비교
- 서빙 아키텍처 패턴 - REST vs gRPC, 동기(Sync) vs 비동기(Async) 처리 방식
- [여기에 아키텍처 다이어그램을 채워 넣으세요]

모델 서빙 아키텍처

DAY1 :: PIPELINE STAGE 2/7

학습 vs 서빙 - 아키텍처 관점 비교

핵심 개념

관점	학습 파이프라인	서빙 파이프라인
실행 환경	GPU 클러스터, 배치 스케줄러	API 서버, 오토스케일링 인프라
코드 성격	실험적, 반복 수정	안정적, 변경에 신중
버전 관리 대상	실험(Experiment), 하이퍼파라미터	배포된 모델 버전, API 스펙
장애 대응	실패 시 재실행	즉시 롤백·Fallback 필요

모델 서빙 아키텍처

DAY1 :: PIPELINE STAGE 2/7

배치 추론 vs 실시간 추론 - 언제 무엇을 쓰는가

핵심 개념

배치 추론 (Batch Inference)

대량의 데이터를 정해진 시간에 한꺼번에 처리

- 야간 리포트 생성
- 대량 고객 세그먼트 분류
- 지연시간보다 처리량이 중요할 때

실시간 추론 (Real-time Inference)

사용자 요청이 오는 즉시 응답

- 챗봇·추천 API
- 이상 거래 탐지
- 즉각적인 응답이 필수일 때

모델 서빙 아키텍처

DAY1 :: PIPELINE STAGE 2/7

서빙 아키텍처 패턴 - 프로토콜 × 처리방식 매트릭스

핵심 개념

패턴	특징	적합한 상황
REST + 동기	구현 간단, 디버깅 쉬움	소규모 API, 프로토타입
REST + 비동기	동시 요청 처리량 개선	이번 과정의 기본 형태 (FastAPI)
gRPC + 동기/비동기	낮은 지연시간, 강타입 스키마	서비스 간 내부 통신, 고성능 요구 환경
WebSocket	지속 연결, 양방향 메시지	채널·대상 탐색, WebRTC 시그널링 중계

이번 과정은 REST + 비동기(FastAPI)를 기본으로 하되, 실시간 채널 연결이 필요한 시나리오를 위해 WebSocket·WebRTC도 다음 슬라이드에서 다룹니다 - 실제 MVP에 필요한 만큼만 다룹니다.

모델 서빙 아키텍처

DAY1 :: PIPELINE STAGE 2/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

WebSocket 기초 - 채널·대상 탐색

예제 코드

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

@app.websocket("/ws/{room_id}")
async def channel_discovery(websocket: WebSocket, room_id: str):
    await websocket.accept()
    try:
        while True:
            data = await websocket.receive_json()
            if data["type"] == "find_peer":
                peer_id = find_available_peer(room_id)
                await websocket.send_json({"type": "peer_found", "peer_id": peer_id})
            elif data["type"] == "sdp_offer":
                await relay_to_peer(data["target"], data)  # 대상 확정 시 SDP 중계
    except WebSocketDisconnect:
        remove_from_room(room_id, websocket)
```

- WebSocket은 연결을 유지한 채 '지금 연결 가능한 대상이 있는지' 탐색하고 신호를 주고받는 역할만 담당 - 미디어 자체는 전송하지 않음

모델 서빙 아키텍처

DAY1 :: PIPELINE STAGE 2/7

WebRTC - 대상 확정 후 P2P 연결로 전환

핵심 개념

- 앞 슬라이드의 WebSocket으로 대상을 탐색하고, 대상이 확정되면 WebRTC로 연결을 전환하는 2단계 흐름
- WebRTC는 브라우저 간 P2P로 오디오·영상·데이터를 실시간 전송하는 웹 표준 기술



FastAPI는 미디어를 전송하지 않고 '연결 수립을 중개'할 뿐입니다. WebSocket 연결(탐색)이 끝나면, 실제 오디오/비디오는 WebRTC로 Client 간 P2P 직접 전송됩니다.

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7



--theory · 1h

- FastAPI 기본 구조 - 라우팅(Routing), 요청/응답 모델 정의
- Pydantic을 이용한 입출력 스키마 정의 - 타입 검증과 데이터 유효성 검사
- 자동 문서화(Swagger UI / OpenAPI) - 별도 문서 작업 없이 API 명세 자동 생성
- [여기에 코드 예제를 채워 넣으세요]

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

FastAPI 최소 서버 - 코드 5줄로 시작하기

예제 코드

```
from fastapi import FastAPI

app = FastAPI(title="Model Serving API")

@app.get("/")
def root():
    return {"status": "ok"}

# 실행 uvicorn main:app --host 0.0.0.0 --port 8088 --reload
```

- FastAPI() 인스턴스 하나가 서버 애플리케이션 전체를 나타냄
- @app.get("/경로")처럼 데코레이터로 URL과 함수를 연결(라우팅)
- uvicorn이 실제로 서버를 구동하는 ASGI 실행기 역할
- 서버 노드에서는 0.0.0.0으로 띄우고 외부 접속 라우팅은 해당 노드 사설 IP로 라우팅, 예시 포트는 8088

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

라우팅 & 요청/응답 모델 - 추론 엔드포인트 설계

예제 코드

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class PredictRequest(BaseModel):
    text: str

class PredictResponse(BaseModel):
    label: str
    score: float

@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    # 모델 추론 로직 (예시)
    return PredictResponse(label="positive", score=0.97)
```

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

Pydantic으로 입출력 스키마 정의 - 입력 검증

예제 코드

```
from pydantic import BaseModel, Field

class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=1000)
    top_k: int = Field(default=1, ge=1, le=10)
```

- text가 비어있거나 top_k가 1~10 범위를 벗어나면 FastAPI가 요청을 아예 함수까지 전달하지 않음
- 대신 자동으로 422 상태코드와 함께 '어떤 필드가 왜 잘못됐는지' 상세 메시지를 반환
- → 개발자가 if문으로 직접 검증 코드를 짤 필요가 줄어듦

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

요청/응답 Schema vs DB Model - 역할이 다르다

핵심 개념

구분	Pydantic (API Schema)	SQLAlchemy (DB Model)
목적	API 입출력 데이터 검증	테이블 구조 정의·매핑
존재 시점	요청마다 생성되는 임시 객체	DB에 영속적으로 저장되는 레코드
대표 도구	Pydantic (BaseModel)	SQLAlchemy (ORM Model)
예시	PredictRequest / PredictResponse	PredictionLog (테이블 매핑 클래스)

같은 '예측 결과'라도 API 응답용 형태와 DB 저장용 형태는 다를 수 있습니다 - 계층을 분리해야 한쪽 변경이 다른 쪽에 영향을 덜 줍니다.

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

MariaDB 컨테이너 연결 설정

예제 코드

```
# MariaDB를 컨테이너로 기동
docker run -d --name mariadb-serving \
  -e MARIADB_ROOT_PASSWORD=changeme -e MARIADB_DATABASE=serving_db \
  -p 3306:3306 mariadb:11
```

```
# FastAPI에서 SQLAlchemy로 연결
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "mysql+pymysql://root:changeme@localhost:3306/serving_db"
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine)
```

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

테이블 매핑을 위한 SQLAlchemy 모델 설계

예제 코드

```
from sqlalchemy import Column, Integer, String, Float, DateTime
from sqlalchemy.orm import declarative_base
from datetime import datetime

Base = declarative_base()

class PredictionLog(Base):
    __tablename__ = "prediction_logs"
    id = Column(Integer, primary_key=True, index=True)
    input_text = Column(String(1000))
    label = Column(String(50))
    score = Column(Float)
    created_at = Column(DateTime, default=datetime.utcnow)
```

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

추론 결과를 MariaDB에 저장하는 통합 엔드포인트

예제 코드

```
from fastapi import Depends
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest, db: Session = Depends(get_db)):
    label, score = model.predict([req.text])[0]
    db.add(PredictionLog(input_text=req.text, label=label, score=float(score)))
    db.commit()
    return PredictResponse(label=label, score=float(score))
```

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

자동 문서화 - Swagger UI / OpenAPI

핵심 개념

- 서버 실행 후 /docs 에 접속하면 별도 작성 없이 API 문서(Swagger UI)가 자동 생성됨
- 함수의 타입 힌트와 Pydantic 모델 정의만으로 요청/응답 스키마가 문서에 그대로 반영
- 문서 화면에서 'Try it out' 버튼으로 실제 요청을 바로 테스트 가능

```
POST  /predict

Request body
{ "text": "string" }

Response 200
{ "label": "string", "score": 0 }
```

FastAPI 기초

DAY1 :: PIPELINE STAGE 3/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

서빙에 필요한 FastAPI 문법만 - 이번 과정의 범위

MVP 스코프

✓ 이번 과정에서 다룬다

- 라우팅 (GET / POST)
- Pydantic 요청·응답 모델
- 상태코드·예외 처리(HTTPException)
- 모델 서빙 처리에 필요한 FastAPI 핵심 기능
- 자동 문서화(Swagger)
- MariaDB 연동(SQLAlchemy ORM)
- WebSocket(채널·대상 탐색) → WebRTC(P2P 전환)

다루지 않는다

- 인증/인가 (OAuth2, JWT)
- 백그라운드 태스크 큐
- HTML 템플릿 렌더링 등
- 기타 FastAPI로 구현 가능한 일반 기능 제외

판단 기준: '완벽한 커버리지'가 아니라 'AIOps 서빙 MVP가 지금 당장 동작하는가' - 필요하면 넣고, 불필요하면 뺍니다.

비동기 처리

DAY1 :: PIPELINE STAGE 4/7



비동기 처리

--theory · 1h

- 비동기 처리(async/await) 이해 - 동시 요청을 효율적으로 처리하는 원리
- 배치(Batching) 처리로 처리량(Throughput) 개선하는 방법
- 동기 vs 비동기 서빙 서버 설계 비교 - 두 방식의 트레이드오프
- [여기에 성능 비교 자료를 채워 넣으세요]

비동기 처리

DAY1 :: PIPELINE STAGE 4/7

동기 vs 비동기 - 무엇이 다른가

핵심 개념

- 동기(Sync) - 한 요청의 추론이 끝날 때까지 서버가 다른 요청을 처리하지 못하고 대기
- 비동기(Async) - 추론(I/O 대기) 중에도 이벤트 루프가 다른 요청을 함께 처리 가능
- 즉, 비동기는 '더 빠른 추론'이 아니라 '동시에 더 많은 요청을 받아낼 수 있는 구조'

서버 방식	동시 요청 수	체감 처리 시간(예시)
동기 서버	3	순차 처리 → 총 3초 소요
비동기 서버	3	동시 처리 → 약 1초 내외로 단축 가능

비동기 처리

DAY1 :: PIPELINE STAGE 4/7

async/await 코드 - 동기 vs 비동기 엔드포인트

예제 코드

```
# 동기 방식 - 추론 중 다른 요청을 처리할 수 없음
@app.post("/predict-sync")
def predict_sync(req: PredictRequest):
    result = heavy_model_inference(req.text)  # blocking
    return result

# 비동기 방식 - 추론 대기 중 다른 요청도 함께 처리
@app.post("/predict-async")
async def predict_async(req: PredictRequest):
    result = await run_in_threadpool(heavy_model_inference, req.text)
    return result
```

비동기 처리

DAY1 :: PIPELINE STAGE 4/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

배치(Batching) 처리로 처리량 개선

예제 코드

```
# 여러 요청을 짧은 시간 동안 모아 한 번에 추론하는 개념 코드
BATCH_SIZE, BATCH_TIMEOUT = 8, 0.05  # 최대 8건 또는 50ms 대기

async def batch_worker(queue):
    batch = await collect_until(queue, BATCH_SIZE, BATCH_TIMEOUT)
    results = model.predict_batch([item.text for item in batch])
    for item, result in zip(batch, results):
        item.future.set_result(result)  # 각 요청자에게 결과 반환
```

- 요청을 하나씩 추론하는 대신 여러 건을 묶어 GPU 연산 1회로 처리 → 처리량(Throughput) 향상

비동기 처리

DAY1 :: PIPELINE STAGE 4/7

동기 vs 비동기 서버 설계 - 트레이드오프

핵심 정리

기준	동기(Sync)	비동기(Async)
구현 난이도	낮음 - 직관적	중간 - async/await 이해 필요
동시 처리량	낮음	높음
디버깅	쉬움	비동기 특유의 디버깅 난이도 존재
적합 상황	CPU 바운드, 단순 로직	I/O 바운드(모델 추론 대기 등)

서빙 안정성

DAY1 :: PIPELINE STAGE 5/7



서빙 안정성

--theory/hands-on · 1h

- 추론 서버 구현 시 필요한 입력 검증(Validation)과 에러 핸들링(Error Handling)
- 헬스체크(Health Check) 엔드포인트 설계 - 서버 상태를 외부에서 확인하는 방법
- 모델 로딩 전략 - Lazy Loading vs Eager Loading 비교, 로깅(Logging) 기초

서빙 안정성

DAY1 :: PIPELINE STAGE 5/7

입력 검증 & 에러 핸들링

예제 코드

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.post("/predict")
def predict(req: PredictRequest):
    if not req.text.strip():
        raise HTTPException(status_code=400, detail="text는 비어있을 수 없습니다")
    try:
        return model.predict(req.text)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"추론 실패: {e}")
```

서빙 안정성

DAY1 :: PIPELINE STAGE 5/7

헬스체크(Health Check) 엔드포인트 설계

예제 코드

```
@app.get("/health")
def health_check():
    return {
        "status": "healthy" if model is not None else "unhealthy",
        "model_loaded": model is not None,
    }
```

- 로드밸런서·오케스트레이터(Kubernetes 등)가 이 엔드포인트로 서버 생존 여부를 주기적으로 확인
- Day3 AIOps 모니터링에서도 이 엔드포인트를 그대로 감시 대상으로 사용

서빙 안정성

DAY1 :: PIPELINE STAGE 5/7

모델 로딩 전략 - Lazy Loading vs Eager Loading

예제 코드

```
# Eager Loading - 서버 시작 시 모델을 즉시 로드
model = None
@app.on_event("startup")
def load_model_eager():
    global model
    model = load_model("model.pt")  # 시작은 느리지만 첫 요청부터 빠름

# Lazy Loading - 첫 요청이 올 때 모델을 로드
def get_model():
    global model
    if model is None:
        model = load_model("model.pt")  # 시작은 빠르지만 첫 요청이 느림
    return model
```

서빙 안정성

DAY1 :: PIPELINE STAGE 5/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

로깅(Logging) 기초 - 요청·응답 추적

예제 코드

```
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("model-serving")

@app.post("/predict")
def predict(req: PredictRequest):
    logger.info(f"request received: text_len={len(req.text)}")
    result = model.predict(req.text)
    logger.info(f"response sent: label={result.label}")
    return result
```

- 요청·응답 로그는 장애 원인 분석뿐 아니라, Day3 AIOps의 지표 수집 기반 데이터가 됩니다

FastAPI 엔드 포인트 실습

DAY1 :: PIPELINE STAGE 6/7



--guided-lab · 1h

- 모델 추론 API 만들기 - 간단한 ML/LLM 모델을 FastAPI 엔드포인트로 래핑
- 요청(Request) → 추론(Inference) → 응답(Response) 흐름을 직접 구현
- 실습 결과를 Swagger UI에서 테스트하며 API 동작 확인

FastAPI 엔드 포인트 실습

DAY1 :: PIPELINE STAGE 6/7

①서빙·AIOps이해 | ②서빙아키텍처 | ③FastAPI기초 | ④비동기처리 | ⑤서빙안정성 | ⑥엔드포인트실습 | ⑦통합실습

실습 시나리오 정의 - 감성분류 모델 서빙

실습 시나리오

목표

간단한 감성분류(Sentiment) 모델을 FastAPI 엔드포인트로 래핑

입력

text: 리뷰 문장 1개

출력

label(positive/negative), score(확률)

1 **모델 파일 준비**
사전 학습된 감성분류 모델(.pkl 등)을 프로젝트에 배치

2 **스키마 정의**
PredictRequest/PredictResponse를 Pydantic으로 정의

3 **엔드포인트 구현**
/predict 라우트에서 모델을 호출해 결과 반환

4 **Swagger로 테스트**
/docs에서 직접 요청을 보내 정상 동작 확인

FastAPI 엔드 포인트 실습

DAY1 :: PIPELINE STAGE 6/7

실습 코드 - 모델을 FastAPI 엔드포인트로 래핑

예제 코드

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()
model = joblib.load("sentiment_model.pkl")

class PredictRequest(BaseModel):
    text: str

class PredictResponse(BaseModel):
    label: str
    score: float

@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    label, score = model.predict([req.text])[0]
    return PredictResponse(label=label, score=float(score))
```

FastAPI 엔드 포인트 실습

DAY1 :: PIPELINE STAGE 6/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

요청 → 응답 흐름 테스트 (curl / Swagger)

예제 코드

```
$ curl -X POST http://localhost:8000/predict \
    -H "Content-Type: application/json" \
    -d '{"text": "이 제품 정말 좋아요!"}'

# 응답
{"label": "positive", "score": 0.95}
```

- 동일한 요청을 /docs 화면의 'Try it out' 버튼으로도 그대로 재현 가능
- 응답 지연시간(ms)을 함께 확인해두면 이후 Day3 모니터링 기준값으로 활용 가능

FastAPI 서버 빌드업 실습

DAY1 :: PIPELINE STAGE 7/7



--capstone-lab · 2h

PART 1

서버 골격 구축

- 라우팅 + 입출력 스키마(입력 검증) 구성
- 에러 핸들링 + 헬스체크 엔드포인트 추가
- 모델 추론 엔드포인트 연결 → Swagger UI로 테스트

PART 2

모델 로딩 전략 실습

- Lazy Loading 방식으로 서버 구성 및 동작 확인
- Eager Loading 방식으로 서버 구성 및 동작 확인
- 두 방식의 시작 시간·응답 지연시간 차이 비교

FastAPI 서버 빌드업 실습

DAY1 :: PIPELINE STAGE 7/7

통합 서버 전체 코드 구조 (예시)

실전 예시

```
serving_app/
├── main.py
│   (FastAPI 앱 진입점)
├── schemas.py
│   (Pydantic 요청/응답)
├── model_loader.py
│   (Lazy/Eager 로딩)
└── requirements.txt
```

```python
# main.py 핵심 골격
app = FastAPI()

@app.on_event("startup")
def startup(): ...  # 모델 로딩

@app.get("/health")
def health(): ...

@app.post("/predict")
def predict(req): ...
```

FastAPI 서버 빌드업 실습

DAY1 :: PIPELINE STAGE 7/7



Swagger UI 테스트 절차

실전 프로세스

1 서버 실행
uvicorn main:app --reload 로 로컬 서버 기동

2 /docs 접속
브라우저에서 http://localhost:8000/docs 접속

3 헬스체크 확인
/health 엔드포인트로 모델 로딩 상태 확인

4 /predict 테스트
Try it out으로 샘플 텍스트 입력 후 응답 확인

5 오류 케이스 확인
빈 문자열 등 잘못된 입력으로 422/400 응답 확인

FastAPI 서버 빌드업 실습

DAY1 :: PIPELINE STAGE 7/7

① 서빙·AIOps 이해 | ② 서빙 아키텍처 | ③ FastAPI 기초 | ④ 비동기 처리 | ⑤ 서빙 안정성 | ⑥ 엔드포인트 실습 | ⑦ 통합 실습

Lazy vs Eager 로딩 - 실습 결과 비교표

실습 결과 양식

측정 항목	Lazy Loading	Eager Loading
서버 시작 시간	빠름	느림 (모델 로딩 대기)
첫 요청 응답 시간	느림 (로딩 지연 포함)	빠름 (이미 로딩 완료)
이후 요청 응답 시간	동일하게 빠름	동일하게 빠름
적합한 상황	빠른 배포·재시작이 중요할 때	첫 요청부터 안정적 응답이 필요할 때

실습 중 두 방식의 실제 시작 시간·응답 시간을 직접 측정해 위 표를 채워보세요.

Day 1 마무리 · Day 2 예고

Day1에서 구축한 FastAPI 추론 서버는 Day2 MLOps 파이프라인(컨테이너화)의 기반이 됩니다.

- FastAPI 서버 구조
- 비동기·안정성 구현
- 추론 엔드포인트 실습
- **Day2 컨테이너화**

DAY 1 → DAY 2

3-DAY ROADMAP

Day 1
모델 서빙 기초

Day 2
MLOps 파이프라인

Day 3
AIOps 운영 체계

2. MLOps 파이프라인 (컨테이너화)

- MLOps 개요
- 모델 버전 관리 및 레지스트리 연동
- CI/CD for ML
- Dockerize
- 데이터/모델 검증
- MLOps 개념 실습
- 버전 관리부터 배포 통합 실습

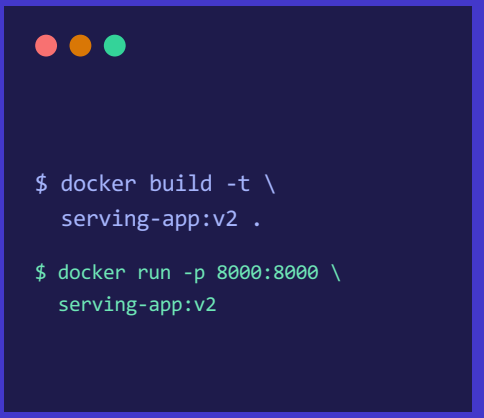
SK 주식회사 AX

DAY 2

모델 서빙 및 AIOps 구성

MLOps 파이프라인을 컨테이너 기반으로 설계·구성하는 과정

교육 기간	**3일 · 24시간 (Day2)**
교육 방식	**이론 50% · 실습 50%**
Day2 목표	**버전관리·컨테이너화 파이프라인 구축**



3-DAY PROGRAM

Day1 → Day2 → Day3

서빙 → 파이프라인 → 운영

MLOps 개요

COURSE :: DAY2 OVERVIEW

교육 개요

Day2의 위치

Day1에서는 AIOps 파이프라인의 개념을 이해하고, 그 서빙 대상이 될 FastAPI 추론 서버의 기초를 다뤘습니다.

Day2에서는 그 추론 모델을 '발견하고 학습시키는' 과정, 즉 MLOps를 다룹니다.

이렇게 만들어진 모델이 결국 Day1 서빙의 대상이 되며, 이후 데이터가 바뀌면(데이터 드리프트) Day1에서 준비한 AIOps가 진가를 발휘하게 됩니다.

3일 로드맵

Day 1 모델 서빙 기초 (FastAPI)
Day 2 MLOps 파이프라인 (컨테이너화)
Day 3 AIOps 파이프라인 (모니터링·자동대응)

교육 기간
3일, 24시간

교육 방식
이론 50% · 실습 50%

필수 선행 과목
Day1 (모델 서빙 및 AIOps 구성)

핵심 DB/도구
MariaDB(컨테이너), Docker

Day2 핵심 산출물

모델 버전 레지스트리 구성 + Day1 서빙 앱 컨테이너화 + 버전별 로딩·배포 통합 실습 완료

DAY2 :: AGENDA

Day 2 목차 한눈에 보기

목차	주요 학습내용	시간	방식
MLOps 개요	DevOps와의 차이점, ML 라이프사이클(데이터→학습→배포→모니터링), MLOps 성숙도 단계	1h	이론
모델 버전 관리 및 레지스트리 연동	모델 레지스트리 개념, 실험 추적(Experiment Tracking), Day1 서빙 API에 버전 파라미터 추가	1h	이론
CI/CD for ML	모델 학습/배포 파이프라인 구조, 컨테이너화(Docker)의 역할, 환경 일관성 문제와 해결	1h	이론
Dockerize	Day1 FastAPI 서빙 앱을 컨테이너 이미지로 빌드, 컨테이너 실행 및 동작 확인	1h	이론/실습
데이터/모델 검증	입력 데이터 스키마 검증, 모델 성능 회귀(regression) 테스트, 배포 전 게이트(Gate) 설계	1h	이론/실습
MLOps 개념 실습	MLOps vs DevOps 차이, ML 라이프사이클 매핑, 오늘 실습의 라이프사이클 단계 확인	1h	이론/실습
버전 관리부터 배포까지 통합 실습	모델 버전 레지스트리 구성 → 서빙 API 버전 파라미터 연동 → Dockerfile 작성·빌드·실행	2h	종합실습

MLOps 개요

Machine Learning의 한계



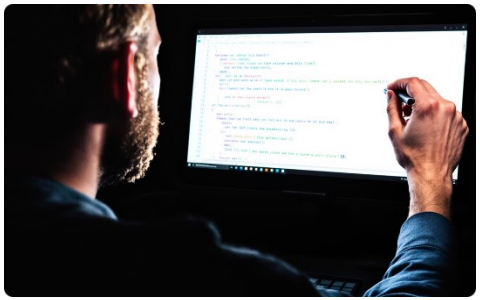
데이터 의존성

데이터 품질에 크게 의존하며, 데이터 편향이나 부족은 모델 성능을 저하시킬 수 있습니다.



해석의 어려움

모델의 예측 결과를 해석하고 이해하기 어려워, 신뢰성 확보에 어려움을 겪습니다.



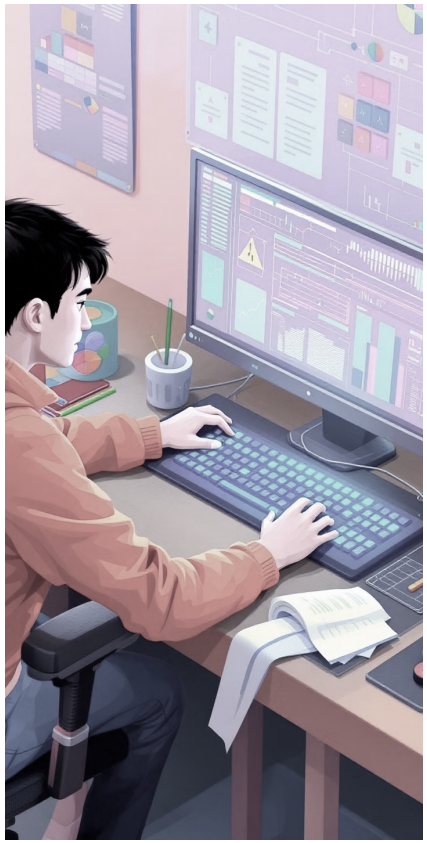
유지보수의 어려움

모델 업데이트 및 유지보수가 복잡하고 시간이 오래 걸려, 지속적인 관리에 어려움을 겪습니다.

MLOps 개요

MLOps란?

MLOps는 Machine Learning(ML) 모델을 개발, 배포 및 관리하는 데 필요한 DevOps 원칙과 관행을 적용하는 것입니다.



MLOps 개요

MLOps의 정의



데이터 과학

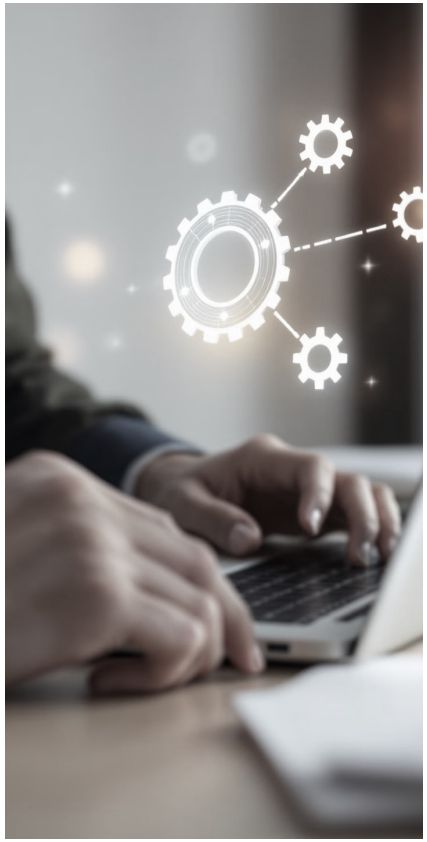
MLOps는 머신러닝(ML) 모델의 개발, 배포, 운영을 위한 DevOps 원칙을 적용하는 방법입니다.

엔지니어링

데이터 과학과 소프트웨어 엔지니어링의 융합을 통해 머신러닝 모델의 전반적인 수명 주기를 효율적으로 관리합니다.

MLOps 개요

MLOps의 목적



모델 개발 속도 향상

MLOps는 개발 프로세스를 자동화하여 모델 개발 속도를 높이고 반복적인 작업을 줄여줍니다.

모델 품질 향상

모델 개발, 배포, 운영 전반에 걸쳐 품질 관리를 강화하여 더 정확하고 신뢰할 수 있는 모델을 구축합니다.

모델 운영 효율성 증대

모델 배포 및 관리를 자동화하고 모니터링 시스템을 구축하여 효율적인 모델 운영을 가능하게 합니다.

MLOps 개요

MLOps 구성요소

데이터 준비	모델 개발	모델 배포	모델 운영
데이터 수집, 정제, 전처리, 라벨링	모델 선택, 학습, 최적화, 평가	모델 패키징, 배포, 모니터링	모델 성능 모니터링, 재학습, 갱신

MLOps 개요

데이터 준비

1

데이터 수집

다양한 출처에서 관련 데이터를 수집합니다.

2

데이터 정제

결측값 처리, 이상값 제거, 데이터 형식 변환 등을 수행합니다.

3

데이터 변환

모델 학습에 적합하도록 데이터를 변환합니다.

4

데이터 분할

학습 데이터, 검증 데이터, 테스트 데이터로 분할합니다.



MLOps 개요

모델 개발



MLOps 개요

모델 테스트 및 검증

1

데이터셋 분할

훈련, 검증, 테스트 데이터셋 분리

2

성능 지표

정확도, 정밀도, 재현율, F1 점수 등 측정

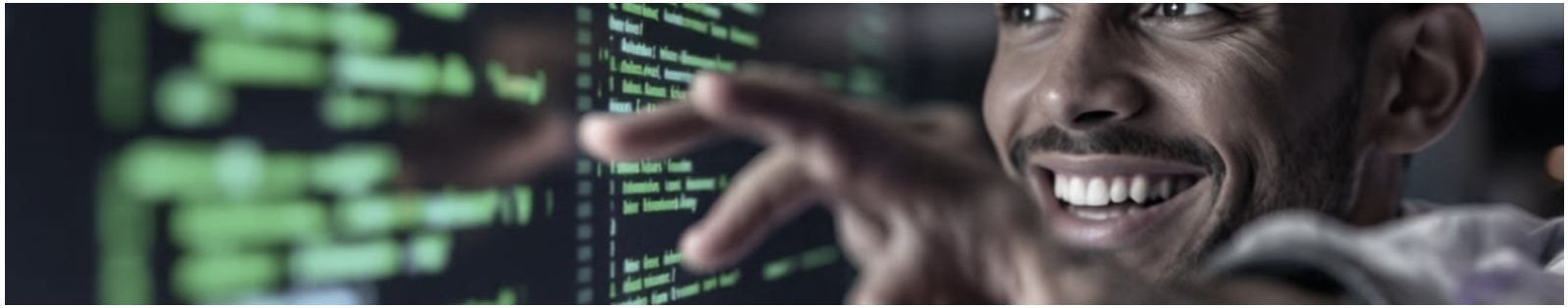
3

모델 검증

과적합 방지, 일반화 성능 확인

MLOps 개요

모델 배포



1

모델 패키징

모델을 배포하기 위해 모델, 코드 및 종속성을 패키징 합니다.

2

인프라 선택

클라우드 또는 온-프레미스 환경과 같은 배포 환경을 선택합니다.

3

배포 자동화

CI/CD 파이프라인을 사용하여 모델 배포를 자동화합니다.

4

데이터 준비

배포된 모델의 성능을 지속적으로 모니터링합니다.

MLOps 개요

모델 모니터링



1

성능 지표 추적

정확도, 정밀도, 재현율, F1 점수 등 모델 성능 지표를 지속적으로 모니터링합니다.

2

데이터 드리프트 감지

입력 데이터의 변화를 감지하여 모델 성능 저하를 예방합니다.

3

모델 오류 분석

모델 오류 발생 원인을 분석하여 모델 개선 방안을 마련합니다.

4

모델 업데이트 알림

모델 성능 저하 또는 오류 발생 시 알림을 통해 빠른 대응을 가능하게 합니다.

MLOps 개요

MLOps 구축 사례

데이터 및 모델 관리

데이터 저장소, 모델 저장소, 버전 관리 시스템 등을 활용하여 데이터와 모델을 효율적으로 관리.

자동화된 모델 학습 및 배포

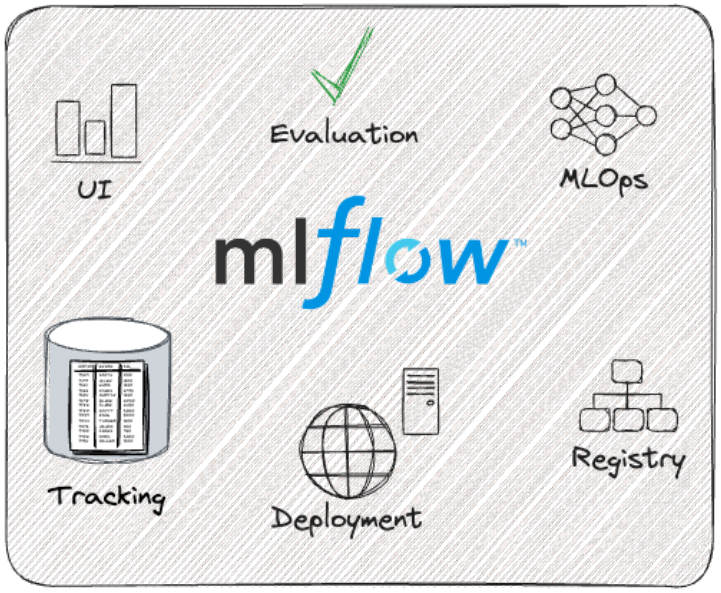
CI/CD 파이프라인을 구축하여 모델 학습, 테스트, 배포 과정 자동화.



MLOps 개요

ML Flow 이해

MLflow는 머신러닝 프로젝트의 실험 관리(Experiment Tracking), 모델 관리(Model Registry), 프로젝트 재현성을 위한 오픈소스 플랫폼



> pip install mlflow
> mlflow --version
> mlflow ui

기본 실행 주소: http://localhost:5001

> mlflow ui --host 127.0.0.1 --port 5001

127.0.0.1 주소 5001번 포트 실행 :
- http://127.0.0.1:5001로 접속 [1]
- 최초 현재 실행 디렉토리 밑에 mlruns 디렉토리 생성

- Tracking : 실험 결과 기록 및 조회
- Projects : 프로젝트 단위 ML코드 패키징
- Models : 모델 버전 관리 및 배포 지원
- Registry : 모델 저장소로써 버전 및 상태 관리

Source : [1] 참고로, Chrome 사용 시 구글 제한 포트 사용 유의 (예, 77, 101, 111, 6000번 등 사용 불가)

MLOps 개요

ML Flow 동작

① 실험 (Experiment) 생성

```
import mlflow

mlflow.set_experiment("New Experiment")
```

② 실험 추적 (Tracking)

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from mlflow.models import infer_signature
import numpy as np

if mlflow.active_run() is not None:
    mlflow.end_run()

# 데이터 준비
X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

# 실험 시작
with mlflow.start_run() as run:
    model = RandomForestRegressor(n_estimators=100)
    model.fit(X_train, y_train)

    score = model.score(X_test, y_test)
# 파라미터, 메트릭 기록
    mlflow.log_param("n_estimators", 100)
    mlflow.log_metric("r2_score", score)

# 입력 예시 추가 (테스트 데이터의 첫 번째 샘플)
    input_example = X_test[:1]

# signature 자동 생성
    signature = infer_signature(X_test, model.predict(X_test))

# 모델 저장 (input_example과 signature 추가)
mlflow.sklearn.log_model(
    sk_model=model,
    artifact_path="model",
    input_example=input_example,
    signature=signature
)

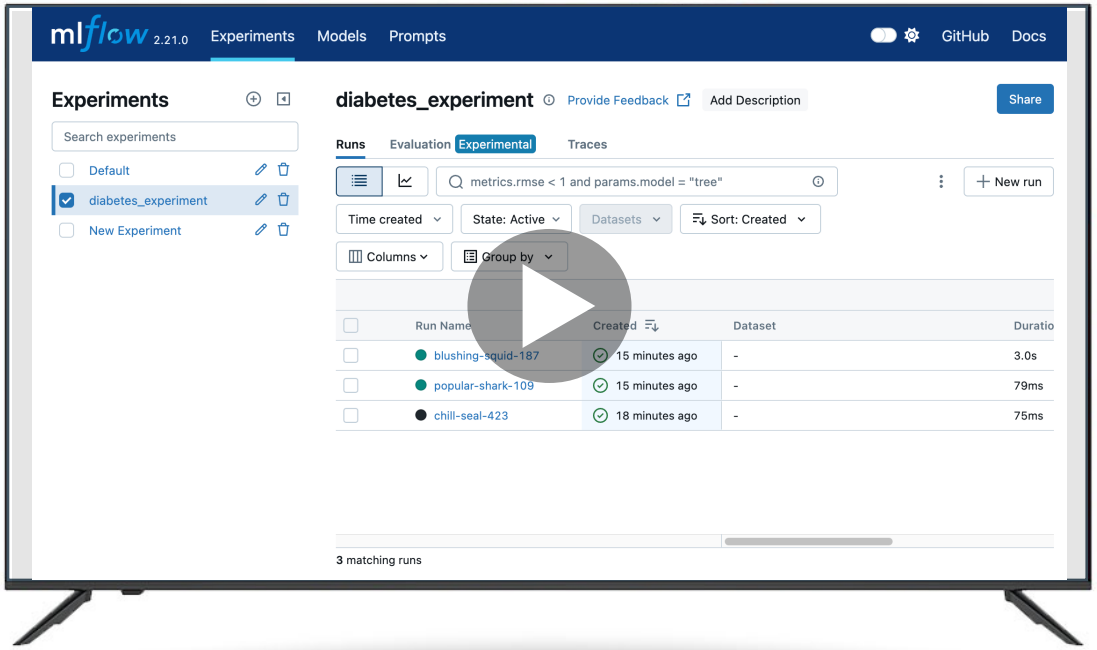
print("MLflow run completed with model, input_example, and signature.")
```

③ 실험 (Experiment) 결과 확인 (UI 실행)

```
> mlflow ui
```

MLOps 개요

ML Flow 활용



MLOps 개요

데이터 및 모델 관리



데이터 저장소

데이터 관리를 위한 데이터 저장소를 구축하여 안전하고 효율적으로 데이터를 저장하고 관리한다.

모델 버전 관리

모델 버전을 추적하고 관리하여 모델 성능을 비교하고 이전 버전으로 롤백할 수 있다.

메타데이터 관리

모델과 관련된 메타데이터를 관리하여 모델 개발, 배포, 모니터링 과정을 투명하게 관리한다.

DAY2 :: 실습 프로젝트

실습 프로젝트 - HumanAI Corporation(HAIC) 주가 예측

가상 종목 HumanAI Corporation(HAIC)의 6개월치 일별 거래 데이터로, 다음날 종가를 예측하는 회귀 모델을 만들고 서빙합니다.

데이터 샘플 미리보기 (일별 시계열, 2026-01-02 ~ 2026-06-30, 총 126 거래일)

Date	Open	High	Low	Close	Volume
2026-01-02	142.50	144.10	141.80	143.20	1,204,500
2026-01-05	143.20	145.00	142.90	144.75	1,150,200
2026-01-06	144.75	146.30	144.00	145.90	1,320,800
...	...	...	...	...	...
2026-06-30	168.40	170.20	167.50	169.85	1,410,300

배포 목표 성능

RMSE ≤ $4.00

예측 종가와 실제 종가의 평균 오차가 4달러를 넘으면 배포 차단

이 프로젝트가 3일간 흐르는 경로

Day2 데이터 생성 → 학습(MLflow Tracking) → RMSE 검증 → 레지스트리(MLflow Registry) → 컨테이너화

Day1 연동 FastAPI 서빙 API에 model_version 파라미터로 연결

Day3 HAIC 데이터 드리프트 감지 → 자동 대응

DAY2 :: MLFLOW ARCHITECTURE

MLOps 파이프라인 구성 - MLflow 아키텍처

이번 과정은 개념 설명에 그치지 않고, 오픈소스 도구 MLflow로 HAIC 프로젝트의 실험 추적과 모델 레지스트리를 실제로 구성합니다.



MLflow 하나로 '실험을 기록하는 것(Tracking)'과 '배포 가능한 버전을 관리하는 것(Registry)'을 함께 다룰 수 있어, 이번 3일 과정의 파이프라인 도구로 채택했습니다.

MLOps 개요

DAY2 :: PIPELINE STAGE 1/7

MLOps 정리

--theory · 1h

- DevOps와의 차이점 - 코드뿐 아니라 HAIC 시계열 데이터의 품질과 모델 성능(RMSE)까지 관리 대상이 됨
- ML 라이프사이클을 HAIC 프로젝트에 대입 - 6개월 일별 데이터 생성 → 회귀모델 학습 → FastAPI 배포 → Day3 모니터링
- MLOps 성숙도 단계 - 수동으로 노트북에서 학습하던 방식에서, 버전관리·컨테이너화된 자동 파이프라인으로 발전
- [여기에 조직별 MLOps 도입 사례를 채워 넣으세요]

MLOps 개요

DAY2 :: PIPELINE STAGE 1/7

MLOps 성숙도 단계 - HAIC로 보는 발전 과정

핵심 개념

단계	특징	비고
Level 0 · 수동	노트북에서 수동 학습, 결과를 파일로 보관	재현 어려움, RMSE 이력 추적 불가
Level 1 · 파이프라인 자동화	MLflow로 실험 자동 기록, Registry로 버전 관리	이번 Day2가 도달하는 단계
Level 2 · CI/CD + 모니터링	새 데이터 유입 시 자동 재학습, Day3 AIOps로 상시 감시	Day3에서 다루는 목표 단계

MLOps 개요

DAY2 :: PIPELINE STAGE 1/7

DevOps vs MLOps - 무엇이 추가되는가

핵심 개념

항목	DevOps	MLOps
코드 버전 관리	공통 (Git)	공통 (Git)
데이터 버전 관리	해당 없음	HAIC 6개월 데이터셋 스냅샷 관리
성능 검증 기준	테스트 통과/실패	RMSE ≤ $4.00 같은 정량적 게이트
산출물	실행 파일/컨테이너	모델 파일 + 실험 메타데이터(MLflow)
재현성 요구	코드만 동일하면 재현	코드+데이터+하이퍼파라미터 모두 동일해야 재현

MLOps 개요

DAY2 :: PIPELINE STAGE 1/7

ML 라이프사이클 - HAIC 프로젝트 단계별 매핑

핵심 개념

데이터 생성 (6개월 일봉) → 학습 (MLflow Tracking) → 검증 (RMSE 게이트) → 배포 (Registry→FastAPI) → 모니터링 (Day3 AIOps)

- Day2에서는 '데이터 생성 → 학습 → 검증 → 배포'까지 4단계를 직접 구축합니다
- 마지막 '모니터링' 단계는 Day3에서 AIOps로 본격 구축하며, 앞의 4단계 위에서 작동합니다

모델 버전 관리 및 레지스트리 연동

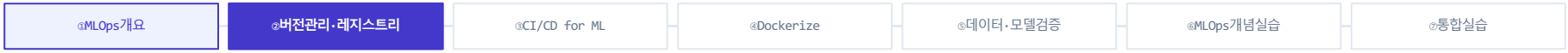
DAY2 :: PIPELINE STAGE 2/7

--theory · 1h

- MLflow Model Registry - HAIC 예측 모델을 v1, v2, v3처럼 버전별로 등록하고 Stage(None → Staging → Production)로 관리
- MLflow Tracking으로 실험 기록 - v1(RMSE $5.20, 기준 미달) → 피처 개선 → v2(RMSE $3.65, Production 승격)
- Day1 FastAPI 기본 구조와를 토대로 MLflow Registry에서 'Production' 단계 모델을 직접 불러오도록 연동

모델 버전 관리 및 레지스트리 연동

DAY2 :: PIPELINE STAGE 2/7



MLflow Tracking 코드 - 실험 기록하기

예제 코드

```
import mlflow
from sklearn.metrics import mean_squared_error

mlflow.set_experiment("haic-price-prediction")

with mlflow.start_run(run_name="v2-feature-improved"):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    rmse = mean_squared_error(y_test, preds, squared=False)

    mlflow.log_param("features", "close,volume,ma5,ma20")
    mlflow.log_metric("rmse", rmse)  # 예: 3.65
    mlflow.sklearn.log_model(model, "model")
```

모델 버전 관리 및 레지스트리 연동

DAY2 :: PIPELINE STAGE 2/7

① MLOps개요 | ② 버전관리·레지스트리 | ③ CI/CD for ML | ④ Dockerize | ⑤ 데이터·모델검증 | ⑥ MLOps개념실습 | ⑦ 통합실습

MLflow Model Registry - 버전 등록과 Stage 전환

예제 코드

```
from mlflow import MlflowClient

# 실험 결과를 Registry에 새 버전으로 등록
result = mlflow.register_model(
    f"runs:/{run.info.run_id}/model", "HAIC_Predictor"
)

# RMSE 게이트를 통과한 버전만 Production으로 승격
client = MlflowClient()
if rmse <= 4.00:
    client.transition_model_version_stage(
        name="HAIC_Predictor", version=result.version, stage="Production"
    )
```

모델 버전 관리 및 레지스트리 연동

DAY2 :: PIPELINE STAGE 2/7

FastAPI에서 Production 모델 로드하기

예제 코드

```
import mlflow.pyfunc
from fastapi import FastAPI

app = FastAPI()

@app.on_event("startup")
def load_production_model():
    global model
    model = mlflow.pyfunc.load_model("models:/HAIC_Predictor/Production")
```

- 'Production' 단계로 승격된 버전만 자동으로 로드 - 별도 버전 번호를 코드에 하드코딩할 필요가 없음
- 새 버전이 Production으로 승격되면 서버 재시작만으로 최신 모델 반영

CI/CD for ML

DAY2 :: PIPELINE STAGE 3/7

CI/CD for ML

--theory · 1h

- 모델 학습/배포 파이프라인 구조 - 새 거래일의 HAIC 데이터가 쌓이면 자동으로 재학습·검증까지 이어지는 흐름
- 컨테이너화(Docker)의 역할 - 학습 시 사용한 pandas/scikit-learn 버전과 서빙 시 버전이 어긋나는 문제 방지
- 환경 일관성 문제와 해결 - '로컬에서는 RMSE $3.8인데 서버에서는 $6.2' 같은 불일치를 컨테이너로 해소
- [여기에 파이프라인 다이어그램을 채워 넣으세요]

CI/CD for ML

DAY2 :: PIPELINE STAGE 3/7

모델 학습/배포 파이프라인 구조

핵심 개념

새 HAIC 거래일 데이터 → 자동 학습 (MLflow Tracking) → RMSE 게이트 체크 → Registry Production 승격 → **컨테이너 재배포**

- RMSE 게이트를 통과하지 못하면 파이프라인이 여기서 멈추고 이전 Production 버전이 계속 서빙됨
- 이 구조 덕분에 '성능이 나쁜 모델이 실수로 배포되는' 상황을 자동으로 방지

CI/CD for ML

DAY2 :: PIPELINE STAGE 3/7

컨테이너화가 해결하는 환경 일관성 문제

핵심 개념

컨테이너 없이

- 로컬: scikit-learn 1.4, 서버: 1.2 - 예측값이 미묘하게 달라짐
- 로컬 RMSE $3.65 → 서버 RMSE $6.20 (재현 불가)
- "내 컴퓨터에서는 됐는데" 문제 반복

컨테이너로 패키징

- requirements.txt로 라이브러리 버전을 이미지에 고정
- 어디서 실행하든 동일한 RMSE 재현
- 학습 환경과 서빙 환경의 완전한 일치

CI/CD for ML

DAY2 :: PIPELINE STAGE 3/7

간단한 CI 파이프라인 스크립트 (예시)

예제 코드

```
#!/bin/bash
# ci_pipeline.sh - 학습부터 배포까지 자동화

python train.py --run-name auto-$(date +%Y%m%d)

RMSE=$(python check_rmse.py)  # MLflow에서 최신 run의 RMSE 조회

if (( $(echo "$RMSE <= 4.00" | bc -l) )); then
    echo "RMSE $RMSE - 게이트 통과, 배포 진행"
    docker build -t serving-app:latest .
    docker run -d -p 8000:8000 serving-app:latest
else
    echo "RMSE $RMSE - 게이트 미달, 배포 중단"
    exit 1
fi
```

Dockerize

DAY2 :: PIPELINE STAGE 4/7



--theory/hands-on · 1h

- Day1에서 만든 FastAPI 서빙 앱 + HAIC 예측 모델(MLflow Registry에서 로드)을 컨테이너 이미지로 빌드
- Dockerfile 기본 구조 - 베이스 이미지, pandas/scikit-learn/mlflow 등 의존성 설치, 앱 코드 복사, 실행 명령
- 필요 시 MLflow Tracking Server도 별도 컨테이너로 함께 구성(docker-compose)해 서빙 앱과 연결

Dockerize

DAY2 :: PIPELINE STAGE 4/7

Dockerfile 작성 - HAIC 서빙 앱 배포 (예시)

예제 코드

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# fastapi, uvicorn, mlflow, scikit-learn, pandas 포함

COPY main.py schemas.py .

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerize

DAY2 :: PIPELINE STAGE 4/7

이미지 빌드 및 컨테이너 실행 (예시)

예제 코드

```
$ docker build -t haic-serving:v2 .
$ docker run -d -p 8000:8000 \
    -e MLFLOW_TRACKING_URI=http://mlflow-server:5000 \
    haic-serving:v2
```

- MLFLOW_TRACKING_URI 환경변수로 컨테이너가 어느 MLflow 서버의 Registry를 바라볼지 지정
- 컨테이너 실행 후 http://localhost:8000/docs 에서 HAIC 예측이 로컬과 동일하게 데모 동작 체크

Dockerize

DAY2 :: PIPELINE STAGE 4/7

④MLOps개요 | ②버전관리·레지스트리 | ③CI/CD for ML | ④Dockerize | ⑤데이터·모델검증 | ⑥MLOps개념실습 | ⑦통합실습

docker compose로 FastAPI + MLflow 함께 구성 (예시)

예제 코드

```
# docker-compose.yml
services:
  mlflow-server:
    image: ghcr.io/mlflow/mlflow
    ports: ["5000:5000"]
    command: mlflow server --host 0.0.0.0

  serving-app:
    build: .
    ports: ["8000:8000"]
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow-server:5000
    depends_on: [mlflow-server]
```

Dockerize

DAY2 :: PIPELINE STAGE 5/7



데이터/모델 검증

--theory/hands-on · 1h

- 입력 데이터 스키마 검증 - HAIC 데이터의 종가·거래량 등이 학습 시점과 같은 형태·범위인지 사전 확인
- 모델 성능 회귀(Regression) 테스트 - 새로 학습한 버전이 이전 버전보다 RMSE가 나빠지지 않았는지 확인
- 배포 전 게이트(Gate) 설계 - RMSE ≤ $4.00을 통과하지 못하면 배포를 자동으로 막는 안전장치

데이터 / 모델 검증

DAY2 :: PIPELINE STAGE 5/7

입력 데이터 스키마 검증 코드 (예시)

예제 코드

```
from pydantic import BaseModel, Field

class HAICFeatures(BaseModel):
    close: float = Field(..., gt=0, description="전일 종가")
    volume: int = Field(..., ge=0)
    ma5: float = Field(..., gt=0, description="5일 이동평균")
    ma20: float = Field(..., gt=0, description="20일 이동평균")

# 범위를 벗어난 값(예: 음수 종가)은 FastAPI가 자동으로 422 응답
```

데이터 / 모델 검증

DAY2 :: PIPELINE STAGE 5/7

모델 성능 회귀(Regression) 테스트 코드 (예시)

예제 코드

```
def test_new_model_not_worse_than_production():
    prod_rmse = get_production_rmse()      # 예: 3.65
    new_rmse = get_latest_run_rmse()        # 예: 3.40

    assert new_rmse <= prod_rmse + 0.5, (
        f"신규 버전 RMSE({new_rmse})가 운영 버전({prod_rmse})보다 "
        "크게 나빠졌습니다 - 배포를 중단합니다"
    )
```

- 절대 기준(RMSE ≤ $4.00)뿐 아니라, 이전 버전 대비 상대적으로 나빠지지 않았는지도 함께 확인

데이터 / 모델 검증

DAY2 :: PIPELINE STAGE 5/7

배포 게이트 설계 - RMSE ≤ $4 자동 체크

핵심 개념

예 (RMSE ≤ $4.00)

학습 완료 → RMSE 계산 → $4.00 이하? → Production 승격 / 배포 차단 (이전 버전 유지)

- 이 게이트는 사람이 수동으로 판단하지 않고, RMSE 수치만으로 자동 결정 - CI 파이프라인(6-3 슬라이드)에 그대로 연결됩니다

MLOps 개념 실습

DAY2 :: PIPELINE STAGE 6/7

MLOps 개념 실습

--guided-lab · 1h

- MLOps vs DevOps 차이를 HAIC 프로젝트로 직접 비교 - '코드 배포'와 'RMSE $4 게이트를 통과한 모델 배포'의 차이
- ML 라이프사이클을 한 장으로 요약 - 데이터생성→학습→RMSE검증→배포→모니터링
- 지금까지 진행한 HAIC 실습(데이터 준비, 버전 등록)이 이 라이프사이클의 어느 단계에 해당하는지 팀별로 매핑

MLOps 개념 실습

DAY2 :: PIPELINE STAGE 6/7

MLOps vs DevOps 실습 비교 (HAIC 예시)

실습 워크시트

기준	일반 DevOps	HAIC 프로젝트(MLOps)
배포 트리거	코드 커밋 (git push)	코드 커밋 + RMSE 게이트 통과
롤백 기준	빌드 실패, 에러율 급증	RMSE 악화, 데이터 드리프트 감지
산출물 저장	컨테이너 이미지 레지스트리	컨테이너 이미지 + MLflow Registry

MLOps 개념 실습

DAY2 :: PIPELINE STAGE 6/7

ML 라이프사이클 한 장 요약

실습 워크시트



MLOps 개념 실습

DAY2 :: PIPELINE STAGE 6/7

① MLOps개요 | ② 버전관리·레지스트리 | ③ CI/CD for ML | ④ Dockerize | ⑤ 데이터·모델검증 | **⑥ MLOps개념실습** | ⑦ 통합실습

매핑 워크시트 (예시)

실습 워크시트

단계	오늘 한 일	진행 상태
데이터	6개월 HAIC 일봉 데이터 준비	완료 (Day2 시작 전 제공)
학습	MLflow Tracking으로 v1/v2 학습·기록	실습 진행
검증	RMSE ≤ $4.00 게이트 확인	실습 진행
배포	Registry Production 승격 + 컨테이너화	실습 진행
모니터링	드리프트 감지·자동 대응	Day3에서 진행

DAY2 :: MLOPS VS AIOPS

MLOps vs AIOps - 무엇이 다른가

구분	MLOps	AIOps
핵심 질문	이 모델을 어떻게 안정적으로 만들고 내보낼까?	지금 서빙 중인 모델이 계속 잘 동작하고 있는가?
주요 활동	데이터 준비, 학습, 검증, 버전관리, 배포 파이프라인	모니터링, 이상 탐지, 자동 대응, 재학습 트리거
다루는 대상	모델 자체 (학습 착수 ~ 배포 시점)	운영 중인 시스템 전체 (인프라·모델·데이터)
시작 시점	모델 개발에 착수하는 순간부터	모델이 서빙되기 시작한 순간부터
이번 과정 매핑	Day 2가 다루는 영역	Day 1에서 기초를 놓고 Day 3에서 본격 구축

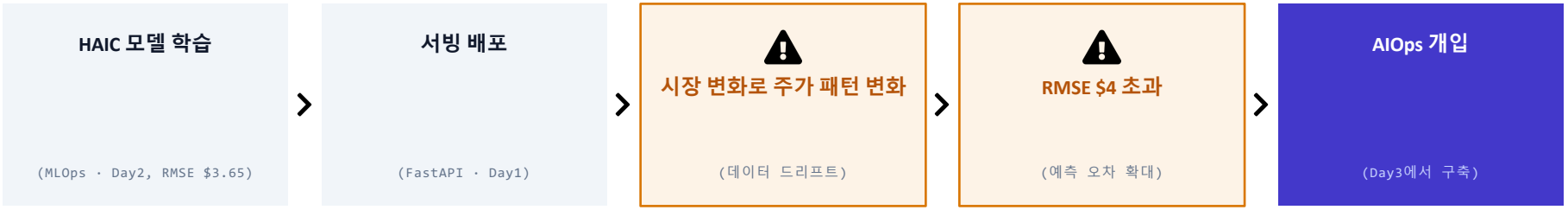
✔ 두 영역은 순차적으로 이어집니다 - MLOps가 모델을 '만들어 내보내는' 일을, AIOps가 그 이후 '계속 잘 돌아가게 지키는' 일을 담당합니다.

MLOps 개념 실습

DAY2 :: WHY AIOPS MATTERS

데이터가 바뀌면 무슨 일이 생기는가?

HAIC 모델은 2026년 상반기(1~6월) 데이터로 학습됐습니다. 하반기에 시장 상황이 바뀌어 주가 패턴이 달라지면(데이터 드리프트), 이 모델의 RMSE는 더 이상 $4 이내를 유지하지 못합니다.



바로 이 지점에서 Day1에서 준비한 AIOps 파이프라인이 빛을 발합니다 - RMSE가 $4를 넘어서는 순간을 자동으로 감지하고, 알림 또는 재학습 트리거로 이어지는 체계가 Day3의 핵심입니다.

MLOps 종합 실습

DAY2 :: PIPELINE STAGE 7/7

버전 관리부터 배포까지 통합 실습

--capstone-lab · 2h

PART 1

MLflow로 HAIC 모델 레지스트리 구성

- mlflow.start_run()으로 학습 실행, log_param/log_metric(rmse)로 실험 기록
- MLflow Model Registry에 모델 등록 후 Production 단계로 승격
- Day1 서빙 API가 Registry의 Production 모델을 로드하는지 확인 (v1 RMSE $5.20 → v2 $3.65)

PART 2

컨테이너 빌드 및 배포

- Dockerfile 작성 (베이스 이미지, 의존성, MLflow 연동 코드 포함)
- 이미지 빌드 → 컨테이너 실행
- Swagger UI로 컨테이너 내부에서 HAIC 다음날 종가 예측 동작 확인

MLOps 종합 실습

DAY2 :: PIPELINE STAGE 7/7

실습 시나리오 최종 정리

실습 시나리오

1. **HAIC 데이터로 v1 모델 학습**
   MLflow Tracking에 기록 → RMSE $5.20 (게이트 미달)
2. **피처 개선 후 v2 재학습**
   이동평균(ma5, ma20) 피처 추가 → RMSE $3.65 (게이트 통과)
3. **v2를 Production으로 승격**
   MLflow Model Registry에서 Stage 전환
4. **FastAPI + Dockerfile 작성**
   Production 모델을 로드하는 서빙 앱을 컨테이너로 패키징
5. **빌드·실행·Swagger 테스트**
   컨테이너 안에서 HAIC 다음날 종가 예측 동작 확인

MLOps 종합 실습

DAY2 :: PIPELINE STAGE 7/7

실습 코드 - MLflow 등록부터 FastAPI 연동까지

예제 코드

```
# train_and_register.py
with mlflow.start_run():
    model.fit(X_train, y_train)
    rmse = evaluate(model, X_test, y_test)
    mlflow.log_metric("rmse", rmse)
    mlflow.sklearn.log_model(model, "model")
    if rmse <= 4.00:
        v = mlflow.register_model(f"runs:/{mlflow.active_run().info.run_id}/model", "HAIC_Predictor")
        MlflowClient().transition_model_version_stage("HAIC_Predictor", v.version, "Production")

# main.py (FastAPI) - 서버 시작 시 Production 모델 자동 로드
model = mlflow.pyfunc.load_model("models:/HAIC_Predictor/Production")
```

MLOps 종합 실습

DAY2 :: PIPELINE STAGE 7/7

실습 체크리스트 및 완료 기준

체크리스트

- MLflow Tracking UI에서 v1, v2 실험의 RMSE를 비교해 확인했는가
- Production 단계로 승격된 버전이 정확히 게이트($4.00)를 통과한 버전인가
- docker build/run 이후 Swagger UI에서 실제 예측 응답이 로컬 실행 결과와 일치하는가
- 컨테이너를 재시작해도 동일한 Production 버전이 로드되는가

Day 2 마무리 · Day 3 예고

Day2에서 만든 버전관리·컨테이너화 파이프라인 위에서, Day3는 실제 모니터링·자동대응 체계를 구축합니다.

- MLOps 라이프사이클 이해
- 버전관리· 레지스트리
- 컨테이너화 (Docker)
- **Day3 AIOps 구축**

DAY 2 → DAY 3

3-DAY ROADMAP

Day 1
모델 서빙 기초

Day 2
MLOps 파이프라인

Day 3
AIOps 운영 체계

3. AIOps 파이프라인 (모니터링·자동대응)

- AIOps 개요
- 모델 모니터링
- 모델 드리프트 & 자동 대응
- 드리프트 감지 시뮬레이션
- 모델 서빙 및 AIOps 파이프라인 가이드
- AIOps 설계
- AIOps 구축

SK 주식회사 AX

DAY 3

모델 서빙 및 AIOps 구성

데이터 드리프트를 자동 감지하고 스스로 재학습·재배포하는 AIOps 파이프라인

교육 기간	**3일 · 24시간 (Day3)**
교육 방식	**이론 50% · 실습 50%**
Day3 목표	**드리프트 자동 감지 → 재학습 → 재배포**



3-DAY PROGRAM

Day1 → Day2 → Day3

서빙 → 파이프라인 → 운영

AIOps 개요

COURSE :: DAY3 OVERVIEW

교육 개요

Day3의 위치

Day1에서 FastAPI로 추론 모델 서빙의 기초를 다졌고, Day2에서는 HAIC 모델을 학습·검증·배포하는 MLOps 파이프라인을 구축했습니다.

Day3에서는 이 서빙 서버에 학습 때와 다른 데이터 패턴을 의도적으로 주입해 RMSE가 $4를 넘도록 만들고, AIOps가 이를 자동으로 감지·재학습·재배포하는 전체 루프를 직접 구현합니다.

3일 로드맵

Day 1 모델 서빙 기초 (FastAPI)
Day 2 MLOps 파이프라인 (MLflow·컨테이너화)
Day 3 AIOps 파이프라인 (드리프트 자동 감지·복구)

교육 기간
3일, 24시간

교육 방식
이론 50% · 실습 50%

필수 선행 과목
Day1·Day2 (모델 서빙 및 AIOps 구성)

실습 대상
Day2에서 배포한 HAIC 서빙 서버

Day3 핵심 산출물

모니터링 지표 API + 드리프트 자동 탐지 로직 + 자동 재학습·재배포 파이프라인 완성

AIOps 개요

DAY3 :: AGENDA

Day 3 목차 한눈에 보기

목차	주요 학습내용	시간	방식
AIOps 개요	MLOps와의 차이(운영 자동화 중심), 모니터링 대상(인프라/모델/데이터), AIOps의 적용 범위	1h	이론
모델 모니터링	추론 지연시간/처리량/에러율 지표, 로그 수집 구조, 알림(Alert) 설계 기초	1h	이론
모델 드리프트 & 자동 대응	데이터 드리프트/컨셉 드리프트 개념, 이상 탐지 기반 자동 알림/재학습 트리거 구조	1h	이론
드리프트 감지 시뮬레이션	입력 분포 변화 데이터를 주입해 이상 탐지 로직 동작 확인, 알림 로그 생성	1h	이론
모델 서빙 및 AIOps 파이프라인 가이드	1~3일 전체 파이프라인 연결(서빙→배포→모니터링→자동대응) 회고, Q&A	1h	이론/실습
AIOps 설계	모니터링 지표의 의미와 측정 방식, 드리프트 종류 구분, 이상탐지→알림→재학습 트리거 구조 작성	1h	개념실습
AIOps 구축	지표 API 추가·집계·경고 → 드리프트 시뮬레이션 → 이상탐지→알림→재학습 트리거 동작	2h	종합실습

DAY3 :: 실습 시나리오

실습 시나리오 - 드리프트 주입부터 자동 복구까지

Day2에서 배포한 HAIC 서빙 서버(RMSE $3.65, Production)를 대상으로, 오늘은 의도적으로 성능을 무너뜨렸다가 AIOps로 스스로 복구시킵니다.



이 루프가 한 번 도는 것으로 끝나지 않고, 이후에도 드리프트가 감지될 때마다 자동으로 반복되어 RMSE $4.00 이내 성능이 지속적으로 유지됩니다.

AIOps 개요

DAY3 :: PIPELINE STAGE 1/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

AIOps 개요

--theory · 1h

- MLOps와의 차이 - 운영 자동화 중심. "이 모델을 어떻게 만들까"가 아니라 "이미 배포된 HAIC 모델이 계속 잘 동작하는가"에 집중
- 모니터링 대상 3가지 - 인프라(컨테이너 상태), 모델(RMSE 등 성능 지표), 데이터(입력 분포 변화)
- AIOps의 적용 범위 - 이번 실습에서는 HAIC 모델의 성능 저하를 자동 감지하고 스스로 복구하는 것까지 다룸

AIOps 개요

DAY3 :: PIPELINE STAGE 1/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

MLOps vs AIOps - Day2/Day3 관점에서 다시 보기

핵심 개념

구분	MLOps (Day2)	AIOps (Day3)
질문	이 모델을 어떻게 만들고 배포할까? (Day2)	배포된 모델이 계속 잘 동작하는가? (Day3)
트리거	코드/데이터 변경, 사람의 실행	RMSE 게이트 초과, 이상 탐지
산출물	MLflow Registry의 Production 버전	자동 감지·알림·재학습 파이프라인
오늘 다루는 대상	-	Day2가 만든 HAIC 서빙 서버 그 자체

AIOps 개요

DAY3 :: PIPELINE STAGE 1/7

모니터링 3대 대상 상세

핵심 개념

인프라

컨테이너 상태, CPU/메모리, 헬스체크 응답

/health 엔드포인트로 확인 (Day1)

모델

RMSE, 예측 분포, 응답 지연시간

예측 로그에서 실제값 대비 오차 계산

데이터

입력 분포(가격 변동성 등)의 변화

최근 입력 통계 vs 학습 시점 통계 비교

AIOps 개요

DAY3 :: PIPELINE STAGE 1/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

AIOps 적용 범위 - 이번 실습에서 구축할 것

핵심 개념

- 포함: 요청/응답 로깅, 지표 집계 API, RMSE 기반 드리프트 탐지, 자동 재학습 트리거, MLflow Registry 재배포 연동
- 제외(범위 밖): Kubernetes 오토스케일링, 별도 알림 채널(Slack/이메일) 연동, 다중 모델 동시 모니터링
- 판단 기준은 Day1과 동일 - '완벽한 운영 체계'가 아니라 'HAIC 드리프트 시나리오가 실제로 자동 복구되는가'

AIOps 개요

DAY3 :: PIPELINE STAGE 1/7

코드 스켈레톤 - AIOps를 위한 FastAPI 앱 구조 확장 (예시)

코드 스켈레톤

```
serving_app/
├── main.py
├── schemas.py
├── model_loader.py
├── routers/
│   ├── predict.py
│   ├── health.py
│   └── metrics.py      # NEW
└── monitoring/         # NEW
    ├── logger.py
    ├── drift_detector.py
    └── retrain_trigger.py
```

```python
# main.py
from fastapi import FastAPI
from routers import predict, health, metrics

app = FastAPI()
app.include_router(predict.router)
app.include_router(health.router)
app.include_router(metrics.router)  # Day3
```

모델 모니터링

DAY3 :: PIPELINE STAGE 2/7

모델 모니터링

--theory · 1h

- 추론 지연시간/처리량/에러율 지표 - HAIC 서빙 API의 응답시간과 에러 발생 여부를 기록
- 로그 수집 구조 - 요청 시각, 입력 데이터, 예측값, RMSE를 구조화된 형태로 로깅
- 알림(Alert) 설계 기초 - 지표가 임계값을 벗어나면 알림이 발생하는 기본 구조
- [여기에 로그 구조 예시를 채워 넣으세요]

모델 모니터링

DAY3 :: PIPELINE STAGE 2/7

추론 지표 정의 - 지연시간·처리량·에러율

핵심 개념

지표	정의	왜 중요한가
지연시간 (Latency)	요청부터 응답까지 걸린 시간(ms)	응답이 느려지면 사용자 경험 저하
처리량 (Throughput)	초당 처리 가능한 요청 수	트래픽 급증 시 서버 여력 파악
에러율 (Error Rate)	5xx 응답이 전체 요청에서 차지하는 비율	서버 장애·모델 오류를 빠르게 인지

모델 모니터링

DAY3 :: PIPELINE STAGE 2/7

로그 수집 구조 - 구조화 로깅

핵심 개념

```
{
  "timestamp": "2026-07-10T09:15:32Z",
  "path": "/predict", "status": 200, "duration_ms": 42,
  "input_close": 168.4, "predicted": 169.1, "actual": null
}
```

- 사람이 읽는 문장형 로그 대신, 파싱하기 쉬운 JSON 형태로 기록 - 이후 집계 스크립트가 그대로 읽을 수 있음
- actual(실제 종가)은 다음날 데이터가 들어와야 채워짐 - RMSE 계산 시 이 필드를 활용

모델 모니터링

DAY3 :: PIPELINE STAGE 2/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

알림(Alert) 설계 기초

핵심 개념

조건	측정 주기	대응 동작
평균 지연시간 > 500ms	5분 집계	경고 로그 출력
에러율 > 5%	5분 집계	경고 로그 출력
RMSE > $4.00	최근 예측 21건 기준	드리프트 알림 + 재학습 트리거

모델 모니터링

DAY3 :: PIPELINE STAGE 2/7

코드 스켈레톤 - 요청 로깅 미들웨어

코드 스켈레톤

```python
# monitoring/logger.py
import time, json, logging
from fastapi import Request

logger = logging.getLogger("aiops")

async def log_requests_middleware(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    duration_ms = round((time.time() - start) * 1000, 2)
    # TODO: request/response 본문에서 필요한 필드 추출
    logger.info(json.dumps({
        "path": request.url.path,
        "status": response.status_code,
        "duration_ms": duration_ms,
    }))
    return response
```

모델 모니터링

DAY3 :: PIPELINE STAGE 2/7

코드 스켈레톤 - /metrics 엔드포인트

코드 스켈레톤

```
# routers/metrics.py
from fastapi import APIRouter

router = APIRouter(prefix="/metrics")

@router.get("/summary")
def get_summary():
    """
    TODO: 로그에서 최근 N분간 평균 응답시간, 에러율, 요청수를 집계
    """
    return {
        "avg_latency_ms": None,  # TODO
        "error_rate": None,      # TODO
        "request_count": None,   # TODO
    }
```

모델 드리프트 & 자동 대응

DAY3 :: PIPELINE STAGE 3/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

모델 드리프트 & 자동 대응

--theory · 1h

- 데이터 드리프트 vs 컨셉 드리프트 - HAIC 주가의 변동성 패턴이 바뀌는 것은 데이터 드리프트의 대표 사례
- 이상 탐지 기반 자동 알림/재학습 트리거 구조 - RMSE가 게이트($4.00)를 넘으면 자동으로 재학습 프로세스가 시작됨
- 이 흐름이 오늘 실습 전체의 핵심입니다 - 앞서 본 시나리오 슬라이드로 이미 전체 그림을 확인했습니다

모델 드리프트 & 자동 대응

DAY3 :: PIPELINE STAGE 3/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

데이터 드리프트 vs 컨셉 드리프트

핵심 개념

구분	데이터 드리프트	컨셉 드리프트
정의	입력 데이터의 분포(통계 특성)가 변함	입력과 정답 사이의 관계 자체가 변함
HAIC 예시	가격 변동성이 갑자기 커진 시장 상황	동일한 지표 조합이라도 오르는 이유가 바뀜 (예 : 실적발표 vs 산업뉴스)
탐지 방법	입력 분포 통계 비교(평균/표준편차)	모델 예측 오차(RMSE) 추이 관찰
오늘 실습 초점	이 유형을 직접 시뮬레이션합니다	개념만 소개

모델 드리프트 & 자동 대응

DAY3 :: PIPELINE STAGE 3/7

이상 탐지 기준 설계 - RMSE 게이트

핵심 개념

- 판단 기준 - 최근 N건(예: 21건)의 예측·실제값으로 RMSE를 계산해 $4.00과 비교
- 너무 짧은 윈도우는 노이즈에 민감하고, 너무 긴 윈도우는 드리프트 반응이 느려짐 - 절충점 필요

```
# monitoring/drift_detector.py
RMSE_THRESHOLD = 4.00
WINDOW_SIZE = 21  # 최근 21건 기준

def is_drift(recent_predictions: list[dict]) -> bool:
    rmse = compute_rmse(recent_predictions)  # TODO: 실제 계산 구현
    return rmse > RMSE_THRESHOLD
```

모델 드리프트 & 자동 대응

DAY3 :: PIPELINE STAGE 3/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

자동 재학습 트리거 구조

핵심 개념

이상 탐지 (RMSE>$4) → 알림 발생 → 최근 3주 데이터 수집 → 재학습 (MLflow) → 게이트 재검증 → **Production 재배포**

- 게이트 재검증에서 통과하지 못하면 Production을 교체하지 않고 기존 버전을 유지 - 안전장치
- 이 구조가 다음 토픽(드리프트 감지 시뮬레이션)에서 실제로 실행됩니다

모델 드리프트 & 자동 대응

DAY3 :: PIPELINE STAGE 3/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

코드 스켈레톤 - 드리프트 감지 → 재학습 트리거 연결

코드 스켈레톤

```
# monitoring/retrain_trigger.py
from monitoring.drift_detector import is_drift

def check_and_trigger(recent_predictions: list[dict]):
    if not is_drift(recent_predictions):
        return {"status": "ok"}

    logger.warning("drift detected - triggering retrain")
    # TODO: 최근 3주 데이터 조회
    # TODO: MLflow로 재학습 실행 (Day2 train_and_register.py 재사용)
    # TODO: 새 RMSE가 게이트를 통과하면 Production으로 승격
    return {"status": "retrain_triggered"}
```

드리프트 감지 시뮬레이션

DAY3 :: PIPELINE STAGE 4/7

드리프트 감지 시뮬레이션

--theory · 1h

- 입력 분포 변화 데이터를 HAIC 서빙 서버에 주입해 이상 탐지 로직이 실제로 동작하는지 확인
- 정상 분포 대비 달라진 입력(예: 변동성이 커진 가격 데이터)을 흘려보내 RMSE 변화를 관찰
- 알림 로그 생성 - RMSE 게이트 초과 시 로그와 알림이 실제로 발생하는지 확인
- [여기에 시뮬레이션 절차를 채워 넣으세요]

드리프트 감지 시뮬레이션

DAY3 :: PIPELINE STAGE 4/7



시뮬레이션 절차 설계

핵심 프로세스

1. **기준 통계 산출**
   학습에 사용한 6개월 HAIC 데이터의 평균·표준편차 계산
2. **정상 입력 테스트**
   같은 분포의 데이터로 예측 → RMSE $4 이내 확인 (베이스라인)
3. **드리프트 데이터 생성**
   변동성을 인위적으로 3배 키운 가격 데이터 생성
4. **드리프트 데이터 주입**
   생성한 데이터를 서빙 서버에 연속 요청으로 전송
5. **결과 관찰**
   RMSE 상승 → 알림 로그 발생 → 재학습 트리거 확인

드리프트 감지 시뮬레이션

DAY3 :: PIPELINE STAGE 4/7

정상 vs 드리프트 데이터 생성 코드

코드 스켈레톤

```
import numpy as np

def generate_normal_batch(n=21, base=165.0, sigma=5.0):
    """학습 데이터와 같은 변동성의 정상 입력"""
    return base + np.random.normal(0, sigma, n)

def generate_drift_batch(n=21, base=165.0, sigma=15.0):
    """변동성을 3배 키운 드리프트 입력 (의도적으로 오차 유발)"""
    return base + np.random.normal(0, sigma, n)

# TODO: 생성한 배치를 /predict 엔드포인트에 순차 요청으로 전송
```

드리프트 감지 시뮬레이션

DAY3 :: PIPELINE STAGE 4/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

실행 결과 예시 - 알림 로그

실행 결과 예시

```
[INFO] batch=normal  rmse=3.71  status=ok
[INFO] batch=drift_injection_start
[WARN] rmse=6.82 > 4.00 - drift detected
[INFO] retrain triggered (window=last_21_days)
[INFO] new_model rmse=3.91 - gate passed
[INFO] production promoted: HAIC_Predictor v3
```

- 이 로그가 곧 Day3 시나리오 슬라이드에서 본, 단계별 루프의 실제 실행 기록입니다

드리프트 감지 시뮬레이션

DAY3 :: PIPELINE STAGE 4/7

코드 스켈레톤 - 시뮬레이션 테스트 엔드포인트

코드 스켈레톤

```
# routers/predict.py 에 추가
from monitoring.retrain_trigger import check_and_trigger

@router.post("/predict/batch-test")
def batch_test(prices: list[float]):
    """
    TODO: 입력된 가격 배치로 연속 예측 실행
    TODO: 예측 기록을 recent_predictions에 누적
    TODO: check_and_trigger(recent_predictions) 호출
    """
    ...
```

모델 서빙 및 AIOps 파이프라인

DAY3 :: PIPELINE STAGE 5/7



모델 서빙 및 AIOps 파이프라인 가이드

--theory/hands-on · 1h

- 1~3일 전체 파이프라인 연결 회고 - 서빙(Day1) → 배포(Day2) → 모니터링·자동대응(Day3)
- AIOps 개요, 모니터링 3대 대상(인프라/모델/데이터), 적용 범위를 다시 한 번 정리
- Q&A - 지금까지 진행한 HAIC 프로젝트에 대한 질의응답

모델 서빙 및 AIOps 파이프라인

DAY3 :: PIPELINE STAGE 5/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

1~3일 전체 파이프라인 다이어그램

회고

Day1 FastAPI 서빙 → Day2 MLflow 학습·배포 → Day3 모니터링·자동대응 → 지속 운영

- 세 날의 산출물이 각각 독립적이지 않고, 하나의 HAIC 서빙 서버 위에 순서대로 쌓였다는 점이 핵심
- Day3 완료 시점부터는 사람이 매번 지켜보지 않아도 마지막 단계(지속 운영)가 스스로 반복됩니다

모델 서빙 및 AIOps 파이프라인

DAY3 :: PIPELINE STAGE 5/7



각 Day 산출물 재확인 체크리스트

회고

Day	산출물	상태
Day1	FastAPI 추론 서버 (라우팅·스키마검증·헬스체크·Swagger)	완료
Day2	MLflow Tracking·Registry, HAIC v2(RMSE $3.65) Production 배포, 컨테이너화	완료
Day3	지표 API, 드리프트 탐지, 자동 재학습·재배포 파이프라인	오늘 완료 목표

모델 서빙 및 AIOps 파이프라인

DAY3 :: PIPELINE STAGE 5/7

Q&A 주요 주제 정리

회고

- 재학습 윈도우(최근 3주)는 왜 이 길이로 정했는가 - 너무 짧으면 노이즈, 너무 길면 반응이 느림
- RMSE 게이트를 통과 못하는 재학습이 반복되면 어떻게 되는가 - 이전 Production 버전이 계속 유지됨
- 실제 운영에서는 재학습을 완전 자동으로 둘 것인가, 사람 승인을 거칠 것인가 - 팀 정책에 따라 다름

모델 서빙 및 AIOps 파이프라인

DAY3 :: PIPELINE STAGE 5/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

코드 스켈레톤 - 지금까지 만든 전체 파일 구조 (예시)

코드 스켈레톤

```
serving_app/
├── main.py                    # Day1
├── schemas.py                 # Day1
├── model_loader.py            # Day1 → Day2(MLflow 연동)
├── Dockerfile, docker-compose.yml   # Day2
├── train_and_register.py      # Day2 (MLflow 학습·등록)
├── routers/
│   ├── predict.py             # Day1 → Day3(시뮬레이션 엔드포인트 추가)
│   ├── health.py              # Day1
│   └── metrics.py             # Day3
└── monitoring/                # Day3
    ├── logger.py
    ├── drift_detector.py
    └── retrain_trigger.py
```

AIOps 설계

DAY3 :: PIPELINE STAGE 6/7



AIOps 설계

--guided-lab · 1h

- 모델 모니터링 지표 - 지연시간/처리량/에러율의 의미와 측정 방식, 구조화 로깅 구조 설계
- 드리프트 & 자동 대응 - 데이터 드리프트 vs 컨셉 드리프트 차이를 팀별로 정리
- 이상 탐지 → 알림 → 재학습 트리거로 이어지는 구조를 팀별로 직접 설계
- [여기에 팀별 설계 워크시트를 채워 넣으세요]

AIOps 설계

DAY3 :: PIPELINE STAGE 6/7

모니터링 지표 설계 워크시트

설계 워크시트

항목	팀 결정값 (직접 작성)	참고
지연시간 임계값	_____	예: 500ms
에러율 임계값	_____	예: 5%
RMSE 게이트	$4.00 (고정)	Day2에서 정한 값 그대로 사용
집계 주기	_____	예: 5분

AIOps 설계

DAY3 :: PIPELINE STAGE 6/7



드리프트 판단 기준 설계 워크시트

설계 워크시트

1 측정 윈도우 크기 결정
최근 몇 건/며칠의 예측을 기준으로 RMSE를 계산할지 (예: 21건)

2 재학습 데이터 범위 결정
재학습 시 최근 며칠 데이터를 사용할지 (예: 최근 3주)

3 재배포 승인 방식 결정
게이트 통과 시 자동 배포할지, 사람 확인을 거칠지

4 실패 시 대응 결정
재학습 후에도 게이트 미달이면 어떤 알림을 보낼지

AIOps 설계

DAY3 :: PIPELINE STAGE 6/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

이상탐지→알림→재학습 트리거 구조 작성

설계 워크시트

- 팀별로 9-1, 9-2에서 정한 값을 바탕으로 전체 흐름도를 화이트보드/슬라이드에 직접 그려봅니다
- 누가(어떤 컴포넌트가) 감지하고, 누가 알림을 받고, 누가/무엇이 재학습을 실행하는지 역할을 명확히 구분
- 다음 토픽(AIOps 구축)에서 이 설계를 그대로 코드로 옮깁니다

AIOps 설계

DAY3 :: PIPELINE STAGE 6/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

코드 스켈레톤 - 팀 설계를 반영한 라우터

코드 스켈레톤

```
# routers/aiops.py - 9-1~9-3에서 정한 설계를 함수 시그니처로 옮기기
from fastapi import APIRouter

router = APIRouter(prefix="/aiops")

@router.get("/drift-status")
def get_drift_status():
    """TODO: 현재 드리프트 상태(정상/의심/확정)를 반환"""
    ...

@router.post("/retrain")
def trigger_retrain():
    """TODO: 팀이 정한 재학습 데이터 범위·승인 방식을 반영해 실행"""
    ...
```

AIOps 구축

DAY3 :: PIPELINE STAGE 7/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

AIOps 구축

--capstone-lab · 2h

PART 1

지표 추가 · 집계 · 경고

- Day2 컨테이너 앱에 요청수/응답시간/에러여부를 로깅하는 API 추가
- 로그에서 평균 응답시간·에러율을 집계하는 스크립트 작성
- 임계값(threshold) 초과 시 콘솔/로그로 경고 출력

PART 2

드리프트 시뮬레이션 & 자동 재학습

- 정상 분포 입력 데이터로 기준 통계 산출
- 분포가 다른 입력(드리프트 데이터)을 주입해 RMSE $4 초과 유도
- 이상 탐지 로직 트리거 → 알림 → 최근 3주 데이터로 재학습 트리거 동작 확인

AIOps 구축

DAY3 :: PIPELINE STAGE 7/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

실습 목표 재확인

실습 시나리오

목표

HAIC 서빙 서버에 지표 수집·드리프트 감지·자동 재학습을 실제로 붙이기

시작 상태

Day2 컨테이너(HAIC v2, RMSE $3.65, Production)가 정상 서빙 중

완료 기준

드리프트 주입 후 자동으로 RMSE $4 이내 새 버전이 재배포되는 것 확인

1 **지표 추가·집계·경고**
다음 슬라이드부터 순서대로 진행

2 **드리프트 시뮬레이션 & 자동 재학습**
PART1 완료 후 이어서 진행

AIOps 구축

DAY3 :: PIPELINE STAGE 7/7

① AIOps개요 | ② 모델모니터링 | ③ 드리프트·자동대응 | ④ 드리프트감지실습 | ⑤ 파이프라인가이드 | ⑥ AIOps설계 | ⑦ AIOps구축

실습 코드 - 지표 로깅 미들웨어 실제 구현

예제 코드

```
# monitoring/logger.py
import time, json, logging

logging.basicConfig(filename="requests.log", level=logging.INFO)
logger = logging.getLogger("aiops")

async def log_requests_middleware(request, call_next):
    start = time.time()
    response = await call_next(request)
    entry = {
        "ts": time.time(),
        "path": request.url.path,
        "status": response.status_code,
        "duration_ms": round((time.time() - start) * 1000, 2),
    }
    logger.info(json.dumps(entry))
    return response

# main.py: app.middleware("http")(log_requests_middleware)
```

AIOps 구축

DAY3 :: PIPELINE STAGE 7/7

실습 코드 - 집계 및 임계값 경고 스크립트

예제 코드

```
# aggregate_metrics.py
import json, time
from statistics import mean

def aggregate(log_path="requests.log", window_sec=300):
    now = time.time()
    rows = [json.loads(l) for l in open(log_path)]
    recent = [r for r in rows if now - r["ts"] <= window_sec]

    avg_latency = mean(r["duration_ms"] for r in recent)
    error_rate = sum(r["status"] >= 500 for r in recent) / len(recent)

    if avg_latency > 500 or error_rate > 0.05:
        print(f"[WARN] latency={avg_latency:.0f}ms error_rate={error_rate:.1%}")
    return {"avg_latency_ms": avg_latency, "error_rate": error_rate}
```

AIOps 구축

DAY3 :: PIPELINE STAGE 7/7

실습 코드 - 드리프트 시뮬레이션 + 재학습 트리거 통합

예제 코드

```
# run_aiops_cycle.py
def run_aiops_cycle(recent_predictions):
    rmse = compute_rmse(recent_predictions)
    if rmse <= RMSE_THRESHOLD:
        return {"status": "ok", "rmse": rmse}

    logger.warning(f"drift detected: rmse={rmse}")
    recent_data = fetch_recent_days(days=21)         # 최근 3주
    new_model, new_rmse = retrain(recent_data)        # Day2 학습 로직 재사용

    if new_rmse <= RMSE_THRESHOLD:
        promote_to_production(new_model)              # MLflow Registry
        logger.info(f"redeployed: new_rmse={new_rmse}")
        return {"status": "retrained", "new_rmse": new_rmse}
    return {"status": "retrain_failed_gate"}
```

AIOps 구축

DAY3 :: PIPELINE STAGE 7/7

실습 완료 체크리스트

체크리스트

- /metrics/summary 호출 시 평균 응답시간·에러율이 실제 값으로 채워지는가
- 정상 데이터로는 RMSE $4 이내, 드리프트 데이터로는 $4 초과가 재현되는가
- 드리프트 주입 후 로그에 [WARN] → [INFO] retrain triggered → [OK] new rmse 순서로 기록되는가
- 재배포 후 /predict 호출 시 새 Production 버전(v3)이 응답하는가
- 이 전체 루프가 재현 가능한 스크립트/명령으로 정리되어 있는가

3일 과정 마무리

HAIC 예측 모델 하나로, 서빙 → 학습·배포 → 운영 자동화까지 AI 서비스의 전체 생애주기를 직접 구축했습니다.

- Day1 - FastAPI 추론 서버 구축
- Day2 - MLflow 학습·배포 파이프라인
- Day3 - 드리프트 자동 감지·복구

지속 가능한 AI 서비스 운영

3-DAY ROADMAP

Day 1
모델 서빙 기초

Day 2
MLOps 파이프라인

Day 3
AIOps 운영 체계

RMSE $4.00
성능 게이트, 끝까지 유지됨

과정 종료